ADVANCED HEAP MANAGER

Sistem za upravljanje memorijom sa rešavanjem
problema heap contention-a

Projekat: Advanced Heap Manager (AHM)

Jezik: C++

Platforma: Windows (HeapCreate API)

Datum: 08.02.2026.

1. SPECIFIKACIJA PROJEKTA

1.1. Zadatak

Napraviti Advanced Heap Manager (AHM) koji rešava problem heap-contention-a. AHM treba da koristi konfigurabilan broj heap-ova i da zahteve za alokacijom obavlja round-robin algoritmom. Takođe treba voditi računa da se prilikom dealokacije memorija vraća u heap iz kog je alocirana.

1.2. API zahtevi

AHM mora implementirati sledeće funkcije:

```
void* Malloc(size_t sizeBytes);
void Free(void* ptr);
```

1.3. Test okruženja

Test aplikacija: Direktno koristi AHM i meri vreme potrebno da se alocira i dealocira 4 GB memorije iz 1, 2, 5, 10 i 50 niti.

Test server: Server prima poruke nasumične veličine od više klijenata i šalje im odgovore nasumične veličine, koristeći AHM za alokaciju bafera.

Oba testa se ponavljaju korišćenjem standardnih malloc() i free() funkcija radi poređenja.

1.4. Opis heap contention problema

Heap contention nastaje kada više niti istovremeno pokušava da alocira ili dealocira memoriju iz deljenog heap-a. Standardni heap implementacije (kao što je Windows default heap) koriste globalni lock za zaštitu internih struktura podataka. Kada nit zatraži alokaciju, ona mora da zaključa ceo heap, izvrši alokaciju, i tek onda otključa heap. Drugi zahtevi moraju da čekaju.

U višenitnim aplikacijama sa intenzivnim alokacijama, ovaj centralni lock postaje usko grlo (bottleneck). Sve niti se takmiče za isti resurs, što dovodi do:

- Serijalizacije: Paralelne niti efektivno izvršavaju alokacije sekvencijalno
- Lock contention-a: Niti troše CPU cikluse čekajući na lock umesto da rade koristan posao
- Context switching overhead-a: OS često prebacuje niti dok čekaju, što dodatno degradira performanse
- Loše skalabilnosti: Dodavanje više niti ne poboljšava performanse, već ih često pogoršava

Ovaj problem je posebno izražen u serverskim aplikacijama i sistemima za obradu podataka gde svaka konekcija ili task alocira memoriju za privremene bafere. AHM rešava ovaj problem raspoređivanjem alokacija na više nezavisnih heap-ova, čime se eliminiše centralno usko grlo i omogućava istinski paralelno izvršavanje.

2. ARHITEKTURA SISTEMA

2.1. Komponente sistema

Komponenta	Odgovornost	Konkurentnost
Heap pool (N heap-ova)	HeapAlloc/HeapFree kroz per-heap lock-ove	Per-heap SRWLOCK
AllocationMap	Mapira ptr → heapId za ispravan Free(ptr)	64 segmenta, svaki sa SRWLOCK
Round-robin counter	Raspoređuje alokacije ravnomerno	atomic<uint32_t>

2.2. Round-robin algoritam

AHM koristi atomični brojač za round-robin raspoređivanje:

```
std::atomic<uint32_t> g_rr{0};
// U Malloc():
const uint32_t ticket = g_rr.fetch_add(1, memory_order_relaxed);
const uint32_t heapId = ticket % heapCount;
```

Ovaj pristup garantuje ravnomernu raspodelu bez dodatne sinhronizacije.

3. IMPLEMENTACIJA - ADVANCED HEAP MANAGER

3.1. Initialize(heapCount, heapSizeBytes)

Inicijalizuje AHM sa N heap-ova:

1. Validacija parametara (heapCount > 0, heapSizeBytes > 0)
2. Alokacija niza HeapEntry struktura (g_heaps)
3. Kreiranje heap-ova pozivom HeapCreate(HEAP_NO_SERIALIZE, heapSizeBytes, 0)
4. Inicijalizacija SRWLOCK za svaki heap
5. Inicijalizacija AllocationMap sa kapacitetom heapCount * 1024 (zaokruženo na stepen dvojke)
6. Postavljanje g_initialized flag-a na true

Napomena: HEAP_NO_SERIALIZE flag je neophodan jer AHM koristi sopstvene lock-ove.

3.2. Malloc(sizeBytes)

Alokacija memorije:

```
void* Malloc(size_t sizeBytes) {
 if (sizeBytes == 0 || !IsReady())
 return nullptr;

 // Round-robin selekcija heap-a
 const uint32_t ticket = g_rr.fetch_add(1, memory_order_relaxed);
 const uint32_t heapId = ticket % g_heapCount;

 HeapEntry& he = g_heaps[heapId];

 // Alokacija sa heap lock-om
 AcquireSRWLockExclusive(&he.heapLock);
 void* ptr = HeapAlloc(he.heapHandle, 0, sizeBytes);
 ReleaseSRWLockExclusive(&he.heapLock);

 if (!ptr) return nullptr;

 // Registracija u AllocationMap
 if (!g_map.Put(ptr, heapId)) {
 // Rollback u slucaju greske
 AcquireSRWLockExclusive(&he.heapLock);
 HeapFree(he.heapHandle, 0, ptr);
 ReleaseSRWLockExclusive(&he.heapLock);
 return nullptr;
 }

 return ptr;
}
```

3.3. Free(ptr)

Dealokacija memorije:

```
void Free(void* ptr) {
 if (!ptr || !IsReady())
 return;

 uint32_t heapId = 0;

 // Lookup i uklanjanje iz mape
 if (!g_map.Take(ptr, heapId))
 return; // ptr nije u mapi

 if (heapId >= g_heapCount)
 return; // neispravan heapId

 HeapEntry& he = g_heaps[heapId];

 // Dealokacija sa heap lock-om
 AcquireSRWLockExclusive(&he.heapLock);
 HeapFree(he.heapHandle, 0, ptr);
 ReleaseSRWLockExclusive(&he.heapLock);
```

```
}
```

Važno: Take operacija prvo uklanja mapiranje, pa tek onda se poziva HeapFree. Ovo sprečava double-free greške.

3.4. Shutdown()

Gašenje sistema:

- Uništavanje AllocationMap strukture (g_map.Destroy())
- Uništavanje svih heap-ova (HeapDestroy za svaki heap)
- Dealokacija g_heaps niza
- Resetovanje brojača i flag-ova

4. ALLOCATIONMAP - DETALJAN DIZAJN

4.1. Zašto AllocationMap postoji

AHM alocira iz više Windows heap-ova. Pokazivač mora biti oslobođen u heap iz kog je alociran. AllocationMap omogućava reverse lookup: ptr → heapId. Ovo omogućava AHM::Free(ptr) da brzo i sigurno locira ispravan heap.

4.2. Model podataka

AllocationMap je open-addressed hash tabela sa linearnim probingom. Svaki segment poseduje sopstvenu tabelu i brojače:

Entry struktura:

Polje	Tip	Opis
key	void*	Alocirani pokazivač
hash	uint64_t	Keširan hash (ubrzava poređenje i rehash)
value	uint32_t	heapId
state	State	Empty / Occupied / Tombstone

Segment struktura:

- SRWLOCK _lock - ekskluzivni lock za segment
- Entry* _entries - niz unosa
- size_t _capacity - kapacitet (stepen dvojke)
- size_t _size - broj zauzetih unosa
- size_t _tombstones - broj Tombstone unosa

4.3. Segmentacija (striping)

Mapa je podeljena na 64 nezavisna segmenta (mora biti stepen dvojke). Selekcija segmenta i indeksiranje unutar segmenta su odvojeni:

```
static constexpr size_t kSegmentCount = 64;
static constexpr size_t kSegmentMask = 63;

// Izbor segmenta (6 najnizih bitova)
SegmentIndex(hash) = hash & kSegmentMask;

// Pocetni indeks u segmentu (ostali bitovi)
SegmentStart(hash, segMask) = (hash >> 6) & segMask;
```

Prvih 6 bitova hash-a biraju segment ($64 = 2^6$), dok preostali bitovi određuju početnu poziciju za probing unutar segmenta.

4.4. Hash funkcija (HashPtr)

HashPtr obavlja dve kritične operacije:

1. Uklanjanje alignment bitova: Pokazivač se pomera udesno za 4 bita, čime se eliminišu najniži bitovi koji su obično nula (16-byte alignment). Ovo sprečava rasipanje entropije.

2. SplitMix64 mešanje: Avalanche mešanje raspoređuje paterne uniformno kroz 64-bitni izlaz. Ovo smanjuje klasterovanje i održava lance probinga kratkim.

```
static uint64_t HashPtr(void* p) {
 uint64_t x = reinterpret_cast<uint64_t>(p);
 x >>= 4; // Uklanjanje alignment bitova
```

```
 // SplitMix64 mesanje
 x ^= x >> 30;
 x *= 0xbf58476d1ce4e5b9ULL;
 x ^= x >> 27;
 x *= 0x94d049bb133111ebULL;
 x ^= x >> 31;
 return x;
}
```

4.5. Linear probing

Svaki segment je nezavisna open-addressing tabela sa linearnim probingom:

```
mask = capacity - 1
start = SegmentStart(hash, mask)
idx = (start + step) & mask, step = 0..capacity-1
```

5. PUT I TAKE OPERACIJE

5.1. Put(ptr, heapId) - Detaljan tok

1. Validacija: _initialized i ptr != null
2. Računanje hash-a van lock-a: hash = HashPtr(ptr)
3. Izbor segmenta: s = _segs[SegmentIndex(hash)]
4. Zaključavanje segmenta (ekskluzivno)
5. EnsureCapacityNoLock(s) - rast ako (size + tombstones + 1) / capacity ≥ 0.85
6. PutNoLock(s, ptr, heapId, hash) - insert/update sa linear probing
7. Otključavanje segmenta

Tombstone reuse strategija:

PutNoLock pamti prvi Tombstone koji naiđe tokom probinga (firstTomb). Ako ne pronađe postojeći ključ (insert operacija), preferira da umetne u firstTomb slot umesto u prazan slot. Ovo održava lance probinga validnim dok vraća kapacitet.

5.2. Take(ptr) - Detaljan tok

1. Validacija: _initialized i ptr != null
2. Računanje hash-a van lock-a
3. Selekcija segmenta: s = _segs[SegmentIndex(hash)]
4. Zaključavanje segmenta (ekskluzivno)
5. LookupNoLock(s, ptr, hash, indexOut, heapIdOut) - probing, rani stop na Empty
6. RemoveAtNoLock(s, index) - označavanje kao Tombstone, dekrementovanje _size, inkrementovanje _tombstones
7. Rebuild ako ShouldRebuildNoLock() vrati true
8. Otključavanje segmenta

5.3. Tombstone strategija i korektnost

Kada se briše element iz open-addressing tabele, nije moguće jednostavno označiti slot kao Empty, jer to može prekinuti lance probinga za ključeve koji su kasnije pomereni. Tombstone čuva integritet lanca: lookup operacije nastavljaju kroz Tombstone, ali mogu sigurno stati na Empty (not found).

5.4. Growth policy (EnsureCapacityNoLock)

Segment raste kada efektivni load factor (occupied + tombstones) pređe 85%:

```
const size_t needed = s._size + s._tombstones + 1;
const bool shouldGrow = (needed * 100) >= (s._capacity * 85);
```

Proces rasta:

- newCap = capacity * 2
- Alokacija newEntries (HeapAlloc, HEAP_ZERO_MEMORY → svi Empty)
- Rehash samo Occupied unosa u novu tabelu koristeći SegmentStart(hash, newMask)
- Oslobađanje stare tabele (HeapFree)
- tombstones = 0

5.5. Rebuild policy (ShouldRebuildNoLock)

Rebuild se aktivira kada tombstone-i značajno smanjuju upotrebljivost. Heuristike aktiviraju rebuild istog kapaciteta kada:

- tombstones ≥ 1024 I
- (tombstones ≥ 25% kapaciteta) ILI
- (tombstones > size) ILI
- ((size + tombstones) ≥ 85% kapaciteta)

RebuildSameCapacityNoLock alocira novu tabelu identičnog kapaciteta, reinsertuje samo Occupied unose, i resetuje tombstones na 0.

5.6. Konkurentnost - Per-segment locking

Ključna karakteristika: Put/Take zaključavaju tačno jedan segment određen hash-om. Dve niti se takmiče samo ako njihovi pokazivači heširaju u isti segment.

Karakteristike:

- Ekskluzivni mod: AcquireSRWLockExclusive za Put i Take (bez shared-mode lookup-a)
- Minimizacija kritične sekcije: Hashing se obavlja pre zaključavanja
- Lokalizacija: Rehash/rebuild utiču samo na jedan segment, ne na celu mapu

5.7. Praktične implikacije

Parametar	Trenutna vrednost	Uticaj
kSegmentCount	64	Povećanje smanjuje contention, ali povećava memory overhead
Load threshold	85%	Viši = manje growths, duži lance probinga Niži = više memorije, češći rehash
Rebuild trigger	tombstones ≥ 1024 I (≥25% ILI >size ILI load≥85%)	Balansirani pristup, sprečava dugačke probe lance

6. TESTIRANJE

6.1. Test aplikacija

Cilj: Merenje performansi direktnog korišćenja AHM vs standardni malloc/free.

Konfiguracija:

- Ukupna memorija: 4 GB
- Veličina bloka: 64 KB
- Broj heap-ova (AHM): 8
- Broj niti: 1, 2, 5, 10, 50

Metodologija:

- Svaka nit alocira bytesPerThread = 4GB / threadCount
- Sve alokacije se prvo obave, zatim sve dealokacije
- Merenje vremena: high_resolution_clock (chrono)
- AHM se inicijalizuje pre testa i gasi posle testa

Kod radnika (worker):

```
static void ahm_worker(size_t bytesPerThread)
{
    constexpr size_t PAGE_SIZE = 4096;
    size_t allocations = bytesPerThread / BLOCK_SIZE;
    size_t remainder = bytesPerThread % BLOCK_SIZE; // ostatak koji ne stane u pun blok
    std::vector<void*> ptrs;
    ptrs.reserve(allocations + (remainder ? 1 : 0)); // rezerviši dodatno mesto za ostatak

    // Alokacija punih blokova
    for (size_t i = 0; i < allocations; i++)
    {
        void* p = AHM::Malloc(BLOCK_SIZE);
        if (!p)
            break;
        // Postavlja 0 u svaku stranicu memorije kako ne bi došlo do lazy alokacije
        char* c = static_cast<char*>(p);
        for (size_t off = 0; off < BLOCK_SIZE; off += PAGE_SIZE)
            c[off] = 0;
        ptrs.push_back(p);
    }

    // Alokacija ostatka ako postoji
    if (remainder > 0) {
        void* p = AHM::Malloc(remainder);
        if (p) {
            char* c = static_cast<char*>(p);
            size_t pages = (remainder + PAGE_SIZE - 1) / PAGE_SIZE;
            for (size_t i = 0; i < pages; i++)
                c[i * PAGE_SIZE] = 0;
            ptrs.push_back(p);
        }
    }

    // Dealokacija
    for (void* p : ptrs)
        AHM::Free(p);
}
```

Napomena o lazy alokaciji:

Alocirana memorija se eksplicitno inicijalizuje pisanjem 0 u prvu lokaciju svake stranice (4096 bajtova). Ovo je neophodno jer operativni sistem obično dodeljuje memoriju lenjivo (lazy allocation) - fizičke stranice se ne dodeljuju dok se ne pristupi. Bez ovog touch-a, merenje vremena bi uključivalo vreme potrebno OS-u da dodeli fizičke stranice tokom prvog pristupa, što nije deo heap manager performansi već OS overhead.

6.2. Test server

Cilj: Simulacija realnog višeklijentskog scenarija sa dinamičkim alokacijama.

Arhitektura:

- Server prima konekcije i kreira po jednu nit za svakog klijenta
- Klijenti šalju poruke nasumične veličine (512 B - 64 KB)
- Server alocira bafer, prima poruku, alocira bafer za odgovor
- Server šalje odgovor nasumične veličine (1 KB - 128 KB)
- Dealokacija bafera posle svake razmene

Arhitekturna napomena:

Server i klijenti su implementirani kao zasebne niti unutar istog procesa radi eliminacije IPC (Inter-Process Communication) overheada i mrežnog overhead-a. Ovaj dizajn omogućava fokusirano merenje heap contention-a bez uticaja eksternih faktora kao što su socket komunikacija, context switching između procesa, ili kernel overhead za IPC mehanizme. Sva komunikacija između servera i klijenata se odigrava u memoriji kroz shared strukture podataka, što izoluje testiranje isključivo na performanse heap manager-a.

Server - ClientHandler:

```
void ClientHandler(SOCKET clientSocket) {
 while (true) {
 int size = 0;
 if (!RecvAll(clientSocket, (char*)&size, sizeof(int)))
 break;

 // Alokacija bafera za prijem
 void* buffer = g_useAHM ? AHM::Malloc(size) : malloc(size);
 if (!buffer) break;

 if (!RecvAll(clientSocket, (char*)buffer, size)) {
 g_useAHM ? AHM::Free(buffer) : free(buffer);
 break;
 }

 // Alokacija bafera za odgovor
 int replySize = rand() % (MAX_SIZE - MIN_SIZE) + MIN_SIZE;
 void* reply = g_useAHM ? AHM::Malloc(replySize)
 : malloc(replySize);
 if (!reply) {
 g_useAHM ? AHM::Free(buffer) : free(buffer);
 break;
 }

 SendAll(clientSocket, (char*)&replySize, sizeof(int));
 SendAll(clientSocket, (char*)reply, replySize);

 // Dealokacija
 if (g_useAHM) {
 AHM::Free(buffer);
 AHM::Free(reply);
 } else {
 free(buffer);
 free(reply);
 }

 g_totalMessages++;
 }
 closesocket(clientSocket);
}
```

Klijent:

```
void RunClient(int id, int numMessages) {
 // ... socket init i connect ...

 for (int i = 0; i < numMessages; ++i) {
 int size = rand() % (MAX_SIZE - MIN_SIZE) + MIN_SIZE;

 // Slanje poruke
 void* data = g_useAHM ? AHM::Malloc(size) : malloc(size);
```

```
    SendAll(sock, (char*)&size, sizeof(int));
    SendAll(sock, (char*)data, size);

    // Prijem odgovora
    int replySize = 0;
    RecvAll(sock, (char*)&replySize, sizeof(int));
    void* reply = g_useAHM ? AHM::Malloc(replySize)
    : malloc(replySize);
    RecvAll(sock, (char*)reply, replySize);

    // Dealokacija
    if (g_useAHM) {
    AHM::Free(data);
    AHM::Free(reply);
    } else {
    free(data);
    free(reply);
    }
    }
    closesocket(sock);
   }
```

Merenje:

- Ukupno vreme izvršavanja (od pokretanja servera do završetka svih klijenata)
- Ukupan broj poruka (atomic counter g_totalMessages)
- Throughput: poruka/sec
- Test se ponavlja jednom sa AHM-om, jednom sa malloc/free

6.3. Očekivani rezultati

Test aplikacija:

Pri malom broju niti (1-2), razlika u performansama je minimalna ili AHM može biti čak sporiji zbog dodatnog overhead-a AllocationMap-a. Sa povećanjem broja niti (5-50), AHM pokazuje sve veće prednosti jer smanjuje heap contention raspoređivanjem alokacija na više heap-ova.

Test server:

U realnom višeklijentskom scenariju sa dinamičkim alokacijama različitih veličina, AHM pokazuje povećan throughput i smanjenu latenciju, posebno pri velikom broju konkurentnih klijenata. Razlika je najizraženija pri 10+ klijenata gde contention postaje dominantan faktor.

7. PREDLOŽENA POBOLJŠANJA

7.1. Motivacija

Trenutna AHM implementacija koristi AllocationMap za održavanje mapiranja ptr → heapId. Iako je AllocationMap segmentiran (64 segmenta sa nezavisnim lock-ovima), i dalje unosi dodatni overhead:

- Heširanja pokazivača
- Lock contention na segment lock-ovima
- Linear probing (potencijalno više pristupa memoriji)
- Upravljanje tombstone-ima
- Dodatna memorija za hash tabelu

Pod velikim višenitnim opterećenjem, AllocationMap može postati usko grlo skalabilnosti.

7.2. Pristup sa zaglavljem alokacije (Allocation Header)

Umesto eksterne mape, metapodaci o vlasništvu se mogu čuvati direktno u malom zaglavlju koje se postavlja neposredno pre memorije vraćene korisniku:

```
[ AllocationHeader | korisnicka memorija ]
  ^                  ^
  AHM interno        vraceno korisniku
```

7.2.1. Sadržaj AllocationHeader

Polje	Tip	Veličina	Opis
heapId	uint32_t	4 bytes	Identifikator heap-a koji je izvršio alokaciju
magic	uint32_t	4 bytes	Validaciona vrednost (0xABCDEF12 ili slično)
		Ukupno 8 bytes	Poravnanje na 16 bytes trivijalno

7.2.2. Modifikovani Malloc flow

1. Izbor heap-a (round-robin)
2. Alokacija sizeof(AllocationHeader) + requestedSize bajtova
3. Inicijalizacija zaglavlja (heapId, magic vrednost)
4. Vraćanje pokazivača na memoriju odmah posle zaglavlja

```
void* Malloc(size_t size) {
 uint32_t heapId = selectHeap();

 size_t totalSize = sizeof(AllocationHeader) + size;
 void* raw = HeapAlloc(heap[heapId], 0, totalSize);

 AllocationHeader* hdr = (AllocationHeader*)raw;
 hdr->heapId = heapId;
 hdr->magic = MAGIC_VALUE;

 return (void*)(hdr + 1); // Korisniku vracamo posle headera
}
```

7.2.3. Modifikovani Free flow

1. Pomeranje pokazivača unazad za sizeof(AllocationHeader)

2. Validacija magic vrednosti (debug builds)
3. Čitanje heapId iz zaglavlja
4. Direktno oslobađanje u heap vlasnik (HeapFree)

```
void Free(void* ptr) {
 if (!ptr) return;

 // Vracanje na pocetak alokacije
 AllocationHeader* hdr = ((AllocationHeader*)ptr) - 1;

 // Debug validacija
 assert(hdr->magic == MAGIC_VALUE);

 uint32_t heapId = hdr->heapId;
 HeapFree(heap[heapId], 0, hdr);
}
```

7.3. Prednosti pristupa sa zaglavljem

✓ Potpuno eliminisanje AllocationMap - nema hash tabele, nema segmenata

✓ O(1) dealokacija bez heširanja ili probinga

✓ Nema dodatnih lock-ova - samo per-heap lock-ovi ostaju

✓ Poboljšana skalabilnost - nema contention-a na AllocationMap

✓ Bolja lokalnost keša - metapodaci su pored korisničkih podataka

✓ Jednostavnija implementacija - manje koda, lakše održavanje

7.4. Nedostaci i izazovi

✗ Memory overhead: 8 bajtova po alokaciji (mali, ali prisutan)

✗ Alignment: Pažljivo rukovanje zahtevima poravnanja

✗ Validacija: Mora se proveriti da li pokazivač potiče iz AHM-a

✗ Double-free detekcija: Potrebne dodatne mere u debug build-ovima

✗ Magic vrednost: Može dati lažno pozitivne/negativne u ekstremnim slučajevima

7.5. Poređenje pristupa

	AllocationMap	**Allocation Header**
Overhead po alokaciji	Entry u hash tabeli (~32 bytes + collision)	8 bytes
Dealokacija	O(1) avg, O(n) worst + hashing + locking	O(1) uvek samo čitanje 8B
Konkurentnost	64 segment locks (potencijalni contention)	Bez dodatnih locks
Memorija	Hash tabela + tombstones (može biti 2x occupied)	Tačno 8B * allocations
Debuggability	Centralizovano praćenje sve alokacije vidljive	Lokalno u header-u teže debagovanje
Implementacija	Kompleksna (hash, probe, rebuild)	Jednostavna (header read/write)

7.6. Reference

Ovaj pristup se uobičajeno koristi u visokoperformansnim alokatorima memorije kao što su:

- jemalloc - Facebook's memory allocator
- tcmalloc - Google's Thread-Caching Malloc
- mimalloc - Microsoft's memory allocator

8. ZAKLJUČAK

Advanced Heap Manager predstavlja efikasno rešenje za problem heap contention-a u višenitnim aplikacijama. Implementacija demonstrira duboko razumevanje problema sinhronizacije i naprednih tehnika za optimizaciju performansi sistema za upravljanje memorijom.

Ključne karakteristike implementacije:

- Multiple heap-ovi: Eliminacija centralnog contention-a raspoređivanjem na N heap-ova
- Round-robin algoritam: Atomični brojač za ravnomernu raspodelu bez dodatne sinhronizacije
- Segmentirana AllocationMap: 64 nezavisna segmenta sa per-segment lock-ovima za minimizaciju contention-a
- Open-addressing sa linear probing: Efikasna hash tabela sa tombstone strategijom
- Adaptivni growth i rebuild: Automatsko upravljanje kapacitetom sa pažljivim threshold-ima

Testiranje:

Sistem je testiran kroz dva komplementarna scenarija - test aplikaciju koja meri čiste performanse alokacije/dealokacije, i test server koji simulira realan višeklijentski rad sa dinamičkim alokacijama. Oba testa pokazuju značajna poboljšanja pri većem broju niti.

Pravci budućeg razvoja:

Najznačajnije predloženo poboljšanje je eliminacija AllocationMap strukture u korist direktnog čuvanja metapodataka u AllocationHeader-u. Ovaj pristup bi:

- Eliminisao sve overhead AllocationMap-a (hashing, probing, lock contention)
- Omogućio O(1) dealokaciju sa jednostavnim čitanjem 8 bajtova
- Smanjio memory footprint
- Pojednostavio implementaciju i održavanje

Tradeoff je minimalan per-allocation overhead (8 bajtova) i nešto teža debuggability, što je prihvatljivo za većinu produkcijskih scenarija gde su performanse prioritet.

Projekat uspešno demonstrira primenu naprednih tehnika konkurentnog programiranja i struktura podataka za rešavanje realnog performansnog problema u sistemskom programiranju.